LESSON BOOKLET • UNIT 1, AREA OF STUDY 1

Data Analysis

Data and information • quality • privacy • databases • SQL • statistics • visualisation

VCE Applied Computing | Study Design 2025– | Outcome 1

Name: _____ **Class:** _____

Teacher: _____ **Year:** _____

How to use this booklet

There is one section for each topic, and every topic works the same way:

Part A — The reading	the new ideas, read at home before the lesson
Part B — Key vocabulary	the words of the topic, defined from that reading
Part C — Key questions	four questions the lesson will start from
Part D — Key knowledge	the reference you annotate in class
Part E — Worked examples	one we do together, then one you do straight after
Part F — Practice	questions like the examples, then extension

Parts A, B and C are done at home, before the lesson, and they are what make the rest work. The reading carries the new ideas, so the lesson can be spent using them instead of being told them. Every key word for the topic appears in **bold** in that reading, and you write its definition straight from the passage — so you arrive already holding the language of the lesson.

You will meet each key word three times on purpose: you define it from the reading, you check it against the slide at the start of the lesson and correct it in another colour, and days later the revision worksheet asks you for it from memory. The fourth lesson of each week is a marimo workbook on the class site. Where a page says *in your booklet*, it means here.

Topic	Title	Page
1	The problem-solving methodology	p. 2
2	Data and information	p. 10
3	Data types and characteristics	p. 17
4	Data quality	p. 24

1 The Problem-solving Methodology

Four stages • requirements • constraints • scope

Key knowledge: **the problem-solving methodology and its four stages**

Part A • Before the lesson — the reading

Do this before the lesson. Read the passage, then answer the four key questions from it. You are not expected to get them all right — they are what the lesson starts from. Bring them with you.

Reading — Why Anyone Would Follow a Method

Give ten people the same problem and nine of them will start by building something. It feels like progress. It is also the single most reliable way to spend three weeks producing a solution to a problem nobody had.

A **methodology** is an ordered way of working through a problem, used the same way every time. It is not about being tidy. It is about doing the thinking in an order where mistakes are still cheap. The problem-solving methodology used in this course has four stages, and they always run in that order.

Analysis comes first, and it is the only stage where you are allowed not to know the answer yet. You work out what the problem actually is, and four things have to come out of it. A **functional requirement** is something the solution must *do* — show the average rainfall for each region. A **non-functional requirement** is something it must *be like* — readable by a Year 7 student, or fast enough to use in a meeting. A **constraint** is something that limits you: the time you have, the software on the laptops, the law, the skill of the people who will use it. **Scope** is where the job stops — what you have decided is not part of this problem. Nothing is built in analysis.

Design decides how. Layouts, charts of what goes in and what comes out, the structure of a database, the plan for a query. Design is still cheap — changing a sketch takes a pencil.

Development is the build, and the testing that goes with it. By now the expensive decisions have already been made, which is exactly the point.

Evaluation is the one people get wrong. It cannot happen on the day you hand the solution over, because at that moment nobody has used it. Evaluation happens later, after the thing has been in use long enough to judge, and it is judged against the requirements written down in analysis — not against whether the client seems pleased.

The two stages people confuse are the first two. Analysis decides *what*. Design decides *how*. If you are drawing a layout, you have left analysis behind, whether you meant to or not.

In Unit 1 you will not write a specification. You will be given one and marked on whether you read it correctly: can you separate what it must do from what it must be like, name what is limiting you, and say where the job stops. That is a skill, and it is practised rather than guessed.

Notes

Part B • Key vocabulary

Every word below appears in **bold** in the reading. Write what each one means *in your own words*, from the reading — the first is done for you. In the lesson you will check them against the slide and correct them in another colour.

Word	What it means, from the reading
Methodology	*An ordered way of working through a problem, used the same way every time.*
Analysis	
Design	
Development	
Evaluation	
Functional requirement	
Non-functional requirement	
Constraint	
Scope	

Notes

Part C • Key questions

Answer these from the reading, in your own words. You are not expected to get them all right — they are what the lesson starts from. Bring them with you.

Key question 1.1

In your own words, what is the difference between the analysis stage and the design stage?

Key question 1.2

The reading says evaluation cannot be done on handover day. Explain why not.

Key question 1.3

Think of something you have made for any subject. Name one thing you did that belonged to *analysis*, and one that belonged to *development*.

Key question 1.4

Which of the four stages do you think gets skipped most often in real life, and what would go wrong as a result?

Part D • Key knowledge

1.1 Four stages, in order

Stage	The question it answers	What you produce
Analysis	What is the problem, and what must the solution do?	Requirements, constraints, scope
Design	How will it work, and what will it look like?	IPO charts, mock-ups, query designs
Development	Build it, and check it works.	The solution itself, and its test results
Evaluation	Did it actually solve the problem?	A judgement, against the requirements, later

1.2 The five kinds of constraint

The reading told you what a constraint is. Naming its *kind* is the skill the examination asks for.

Kind	Examples
Economic	Cost, and time — time is the one that bites at school
Technical	Processing speed, capacity, equipment available, compatibility, security
Social	The level of expertise your users actually have
Legal	Intellectual property, ownership of data, privacy of data
Usability	Accessibility, usefulness, ease of use

Annotate as we go

Add your own example beside each kind during the lesson. Yours will stick better than mine.

Notes

Part E • Worked examples

Worked Example 1.1 — Pulling a brief apart

"The sustainability group wants a one-page summary of Victorian rainfall for 2025, built from Bureau of Meteorology records for six weather stations. It must be understandable at a glance by students in Years 7 to 12. It will be built in a spreadsheet, because that is what is on the school laptops. It is due in two weeks. Forecasting is not wanted."

Functional: show rainfall for 2025 by region, from the six stations' records.
Non-functional: understandable at a glance by a Year 7 to 12 reader.
Constraints: spreadsheet only (technical); two weeks (economic).
Scope: 2025 only; forecasting is out.

Notice that "built in a spreadsheet" is a constraint, not a requirement. The brief is not asking for a spreadsheet — it is telling you that is all you have.

We do 1.1 — A second brief

"The canteen wants to know which days sell out of hot food. Use last term's till records. The report has to be one page the manager can read without training, and it must not name any student. It has to be ready before the next ordering round on Friday."

(a) One functional requirement:

(b) One non-functional requirement:

(c) Two constraints, naming the kind of each:

(d) One thing that is out of scope:

Notes

Part F • Practice

Task 1.1

Write the stage of the problem-solving methodology each activity belongs to.

Activity	Stage	How you know
Interviewing the client about what they need	*Analysis*	*Finding out what the problem is, before a solution exists.*
Writing the SQL that builds the database		
Sketching a mock-up of the dashboard on paper		
Testing that a query returns the right rows		
Asking three months later whether it helped		

Task 1.2

A brief says the solution must be “user friendly”.

(a) Explain why that is not yet a requirement.

(b) Rewrite it as a requirement that could actually be checked.

Task 1.3

Name the kind of each constraint.

Constraint as stated in a brief	Kind
"You have two lessons to finish this."	*Economic*
"It must run on a school laptop with nothing installed."	
"The office staff have never written a formula."	
"The source data may not be redistributed."	
"A screen reader has to be able to read the results."	

Extension 1.4

A solution meets every functional requirement and none of the non-functional ones. Describe what using it would be like, and explain why evaluation would still mark it down.

Extension 1.5

Give one requirement that could reasonably be read as either functional or non-functional, and explain the ambiguity.

Notes

2 Data and Information

What separates them • qualitative and quantitative • identifiers

Key knowledge: **types and purposes of data and information**

Part A • Before the lesson — the reading

Do this before the lesson. Read the passage, then answer the four key questions from it. You are not expected to get them all right — they are what the lesson starts from. Bring them with you.

Reading — The Number That Means Nothing

Here is a number: **24.4**. You cannot do anything with it. You do not know what it measures, where it came from, when, or in what units. It is **data** — raw, unorganised facts and figures that carry no meaning on their own.

Now: *Mildura's mean maximum temperature in 2025 was 24.4* °C. Nothing about the number changed. What changed is that it has been organised, given a context and made to answer a question. That is **information**.

The test is simple. Ask whether it answers a question on its own. If you still have to say what it is about, where it came from or what it is measured in, you are holding data. This matters more than it sounds, because almost everything a computer stores is data, and almost everything a person wants is information. The whole of this area of study lives in the gap between the two.

Data comes in two kinds. **Quantitative** data is counted or measured, and arithmetic on it means something — temperatures, rainfall, how many days it rained. **Qualitative** data describes a quality or a category, and arithmetic on it does not — station names, regions, the state something is in.

Then there is the trap. Some data is stored as digits but is not a number at all. Weather station 76031 is not ten thousand more of anything than station 66031. A postcode, a phone number, a student number: all digits, none of them quantities. These are **identifiers** — values that label a thing rather than measure it. A computer will happily average a column of identifiers and hand you a number. The number is meaningless, and nothing in the system will warn you.

It is also worth knowing where data comes from, because the source shapes what you can trust it for. **Sensor data** is recorded automatically by an instrument — a thermometer, a rain gauge, a step counter, a door scanner. It arrives in huge quantities, it is consistent, and it carries no opinion; but a sensor measures only what it was pointed at, and it keeps recording when it drifts out of calibration. **Demographic data** describes people and populations — age, postcode, language spoken at home, household income. It answers questions sensors cannot, and it is collected from people, which means somebody chose the questions, somebody chose who to ask, and some people did not answer.

The rule worth carrying out of this reading: before you do arithmetic on a column, ask whether adding two of its values together would mean anything. If it would not, you are holding labels, not quantities.

Part B • Key vocabulary

Every word below appears in **bold** in the reading. Write what each one means *in your own words*, from the reading — the first is done for you. In the lesson you will check them against the slide and correct them in another colour.

Word	What it means, from the reading
Data	*Raw, unorganised facts and figures. On their own they carry no meaning.*
Information	
Qualitative	
Quantitative	
Identifier	
Sensor data	
Demographic data	

Notes

Part C • Key questions

Answer these from the reading, in your own words. You are not expected to get them all right — they are what the lesson starts from. Bring them with you.

Key question 2.1

What has to be added to data before it becomes information?

Key question 2.2

Station 76031 is stored as digits. Explain why it is not quantitative data.

Key question 2.3

Give one example of *sensor data* and one of *demographic data* that your school already holds about you.

Key question 2.4

The reading says a bare number cannot be checked. Explain why having the context makes data checkable.

Part D • Key knowledge

2.1 The test

Data or information?

Does this answer a question on its own? If it needs you to say what it is about, where it came from, or what it is measured in, it is still data.

Task 2.1

Write one sentence of your own explaining what has to happen to data before it becomes information.

2.2 Classifying a field

Task 2.2

For each field of a weather observation, write Q (qualitative) or N (quantitative), and say how you decided.

Field	Q or N	How you decided
station_name	*Q*	*A label. Averaging two station names is meaningless.*
max_temp_c		
region_name		
rainfall_mm		
station_id		

Part E • Worked examples

Worked Example 2.1 — Turning data into information

A sensor logs **0.0, 1.2, 18.6, 0.0, 7.4** for five days at one station.

As it stands this is data: five numbers, no units, no place, no dates.

Organise it — these are daily rainfall totals in millimetres at Cape Otway, 1 to 5 May — and it becomes information: *Cape Otway had rain on three of the five days, and 18.6 mm of it fell on one day.*

The number 18.6 did not change. It now answers a question.

We do 2.1 — Your turn

A school sensor logs **8.4, 12.1, 9.7, 8.4, 11.9** for one week.

(a) What is missing before these numbers could be called information?

(b) Invent the missing context, then write one sentence of information from them.

Notes

Part F • Practice

Task 2.3

Decide whether each item is data or information, and say why.

Item	Data or information	Why
(a) 24.4	*Data*	*A bare number with no context.*
(b) Cape Otway had rain on 113 days last year		
(c) 76031		
(d) The wettest month at Cape Otway was July		

Task 2.4

A weather station records **50.0** in its elevation field.

(a) Is this data or information? Say why.

(b) Write a sentence that turns it into information.

Task 2.5

Sensor data and demographic data fail in different ways.

(a) Give one thing that can go wrong with sensor data that cannot go wrong with demographic data.

(b) Give one thing that can go wrong with demographic data that cannot go wrong with sensor data.

Extension 2.6

Postcodes, phone numbers and student numbers are all stored as digits. Explain what they have in common with station_id, and write the rule you would give a junior programmer for deciding whether a numeric field is really a number.

Notes

3 Data Types and Characteristics

Choosing a type • integer against floating point • structure and context

Key knowledge: **types of data, their characteristics, and the data types used to store them**

Part A • Before the lesson — the reading

Do this before the lesson. Read the passage, then answer the four key questions from it. You are not expected to get them all right — they are what the lesson starts from. Bring them with you.

Reading — The Decision Nobody Tells You About

Every time you create a field in a database or a column in a spreadsheet, you make a promise about what can go in it and what can be done with it. That promise is the *data type*, and getting it wrong is one of the few mistakes in this subject that produces no error message at all.

The types you need are few. A **character** is a single letter, digit or symbol, stored as text — a grade of A, a flag of Y. A **string** holds text of any length: names, labels, codes, anything you will never do arithmetic on. An **integer** holds a whole number with no fractional part — a count of days, a year, a humidity reading in whole percent. A **floating point** number holds a number that has a fractional part, which is what every measurement needs: a temperature of 24.4, a rainfall of 0.2, a latitude of -34.2358. A **date/time** holds a point in time, stored so that sorting and filtering by date actually work instead of sorting alphabetically. A **Boolean** holds one of exactly two values, true or false, and nothing else — no "maybe", no blank.

Integer and floating point are the pair that catches people, because to you they are both just numbers. To a database they are different things. Store 24.4 °C in an integer field and the database keeps 24. It does not complain. It does not warn you. The 0.4 is simply gone, and you will not find out until somebody notices your averages are low.

A second silent problem is the difference between zero and empty. A rainfall reading of 0.0 means somebody walked to the gauge, looked, and recorded no rain — that is a measurement. An empty cell means nobody recorded anything — that is a gap. If a program quietly fills the gaps with zeros, every average it reports afterwards is dragged down, and the data now claims that days nobody measured were dry.

Two further words describe whether data is usable at all. **Structure** is the shape data is held in: one row per observation, one column per field, the same type all the way down a column. Structured data can be sorted, filtered and queried by a machine; unstructured data — a paragraph, a photograph, a voice recording — has to be read or watched by a person. **Context** is everything that says what the data is about: the units, the place, the time, who collected it and how. A perfectly structured column with no context cannot be checked, compared or trusted by anyone.

Carry this away: choosing a data type is not a formality, and a column that mixes a number with its unit — "24.4 °C" typed as text — has stopped being a number at all.

Part B • Key vocabulary

Every word below appears in **bold** in the reading. Write what each one means *in your own words*, from the reading — the first is done for you. In the lesson you will check them against the slide and correct them in another colour.

Word	What it means, from the reading
Character	*A single letter, digit or symbol, stored as text.*
String	
Integer	
Floating point	
Date/time	
Boolean	
Structure	
Context	

Notes

Part C • Key questions

Answer these from the reading, in your own words. You are not expected to get them all right — they are what the lesson starts from. Bring them with you.

Key question 3.1

What does choosing a data type actually decide about a field?

Key question 3.2

Explain why storing 24.4 as an integer is worse than it first looks.

Key question 3.3

Explain the difference between a rainfall reading of 0.0 and an empty rainfall cell.

Key question 3.4

Structured data can be queried; unstructured data has to be read. Give one example of each from somewhere in your school.

Part D • Key knowledge

3.1 Choosing the type

Type	Holds	Use it when
Character	One letter, digit or symbol	A single flag or grade
String	Text of any length	Names, labels, identifiers, codes
Integer	A whole number	Counts, years, humidity as a percentage
Floating point	A number with a fractional part	Measurements: temperature, rainfall, latitude
Date/time	A point in time	Anything you will sort or filter by date
Boolean	True or false, nothing else	A yes/no fact with no third option

Task 3.1

A column holds **24.4 °C** — the number and the unit together, as text. Give two separate problems this causes.

Notes

Part E • Worked examples

Worked Example 3.1 — What filling the gaps costs

A month has 30 days: 10 wet days totalling 60 mm, 15 dry days recorded as 0.0, and 5 days with nothing recorded at all.

Mean over the 25 *measured* days: $60 \div 25 = 2.4$ mm.
Mean if the 5 gaps are filled with zeros: $60 \div 30 = 2.0$ mm.

The second figure is lower, and it is wrong — it reports as "dry" five days nobody looked at. Notice the direction: filling gaps with zeros always drags a mean *down*.

We do 3.1 — Your turn

A humidity field holds **0** on one row and is **empty** on another.

(a) State what each one means.

(b) A program replaces every empty cell with 0. Say whether the reported mean humidity goes up or down, and explain why.

Notes

Part F • Practice

Task 3.2

Give the data type you would store each value as, and say why.

Value as recorded	Data type	Why that type
-34.2358	*Floating point*	*A latitude is meaningless without the fractional part.*
VIC		
113		
2025-06-01		
0.0		
yes — the gauge was serviced		

Task 3.3

Humidity is recorded as a whole number from 0 to 100. Someone suggests storing it as text so that "missing" can be typed in.

(a) Give one reason that is a bad idea.

(b) Suggest what should be done with a missing humidity reading instead.

Extension 3.4

A field holds a student's year level. Argue for storing it as an integer, then argue for storing it as a string, and say which you would choose and why.

Notes

4 Data Quality

Five factors • accurate against reliable • bias • outliers

Key knowledge: **factors affecting the quality of data and information**

Part A • Before the lesson — the reading

Do this before the lesson. Read the passage, then answer the four key questions from it. You are not expected to get them all right — they are what the lesson starts from. Bring them with you.

Reading — A Beautiful Chart of Bad Data

Every chart you draw this term inherits the quality of the data underneath it. That cuts one way only: a careful, well-labelled, handsome visualisation of bad data is *worse* than no visualisation at all, because people believe it.

Five factors decide whether data can be trusted. **Accuracy** asks whether the value is correct — a maximum of 85 °C in Mildura is not. **Bias** asks whether the method used to collect the data favoured one answer over the truth. **Integrity** asks whether the data is whole and internally consistent — duplicate rows and three date formats in one column both break it. **Relevance** asks whether the data bears on the question you were actually asked; humidity is irrelevant when you were asked about rainfall, however good the humidity figures are. **Reliability** asks whether the source can be depended on, over and over.

Two of those are constantly confused. Accurate means close to the true value; it is a property of *this reading*. Reliable means dependable over time; it is a property of *the source*. The difference is not academic. A thermometer that reads exactly 2 °C too high every single day is reliable and not accurate — and because it is reliable you can simply subtract two and use it. A thermometer that is right on average but wildly wrong on random days is accurate on average and completely useless, because you cannot tell which days to trust.

Bias is the one students most often misunderstand, because the word suggests somebody is lying. It does not mean that. Bias is built into the method before a single value is recorded. Survey at lunchtime and you miss everyone in class. Use an online form and you miss everyone without reliable internet. Put your weather stations at airports — flat, open, paved ground — and you will read warmer than the suburbs around them.

Two faults turn up in almost every real extract. An **outlier** is a value far away from the rest of the data: a rainfall reading of 487 mm in a column where nothing else passes 50. Some outliers are mistakes and some are the most interesting day of the year, so an outlier is a reason to investigate, never a reason to delete. A **missing value** is a field where nothing was recorded at all — an empty cell, not a zero. Missing values are easy to overlook precisely because they look like nothing, and every average computed over them is quietly wrong.

Finish on this. Five of the six stations in our dataset are at airports. The dataset is not wrong; every reading is a real reading. What would be wrong is the *claim* that these six numbers describe "Victoria's temperature". Most bad data analysis is not bad numbers. It is a true number attached to a claim it cannot support.

Part B • Key vocabulary

Every word below appears in **bold** in the reading. Write what each one means *in your own words*, from the reading — the first is done for you. In the lesson you will check them against the slide and correct them in another colour.

Word	What it means, from the reading
Accuracy	*Whether the value is correct. A maximum of 85 °C in Mildura is not.*
Bias	
Integrity	
Relevance	
Reliability	
Outlier	
Missing value	

Notes

Part C • Key questions

Answer these from the reading, in your own words. You are not expected to get them all right — they are what the lesson starts from. Bring them with you.

Key question 4.1

Explain the difference between *accurate* and *reliable* in your own words.

Key question 4.2

Why is a thermometer that is always 2 °C too high more useful than one that is right on average?

Key question 4.3

The reading says bias is not someone lying. What is it, then?

Key question 4.4

Who is harmed by a beautiful chart of bad data, and why is it worse than no chart at all?

Part D • Key knowledge

4.1 Five factors

Factor	The question	Fails when
Accuracy	Is the value correct?	A maximum of 85 °C in Mildura
Bias	Does the method favour an out-come?	Only surveying students already in the canteen queue
Integrity	Is the data whole and consistent?	Duplicate rows; three date formats in one column
Relevance	Does it bear on the question?	Humidity, when you were asked about rainfall
Reliability	Can the source be depended on?	A sensor that drifts; an anonymous blog

Whose fault is it?

Five of our six stations are at airports. The *dataset* is not wrong — every reading is real. What would be wrong is the *claim* that these six numbers describe "Victoria's temperature".

Notes

Part E • Worked examples

Worked Example 4.1 — Naming the fault

A row of a raw extract reads:

```
99999 | Unknown Station | 2025-03-12 | 82.4 °C | -3.1 | 180
```

82.4 °C — the unit is inside the value, so the column is now text. *Integrity.*
82.4 — not a possible Victorian maximum. *Accuracy.*
-3.1 — rainfall cannot be negative. *Accuracy.*
180 — humidity cannot exceed 100 %. *Accuracy.*
99999 — no such station, so the row points at nothing. *Integrity.*

No real row is this bad. This one was built to carry every fault at once.

We do 4.1 — Your turn

A row reads:

```
86338 |   Essendon Airport   | 12/03/2025 | 23.8 | (empty) | 58
```

(a) Name two faults in this row.

(b) For each, name the quality factor it damages.

Notes

Part F • Practice

Task 4.1

Name the group each collection method leaves out, and which way that pushes the result.

Collection method	Who is left out	Which way it pushes
A survey run in the canteen queue at lunchtime	*Everyone in class, and every-one who brings lunch.*	*Towards "students love the canteen".*
An online form sent by email		
A phone poll on a weekday, 9 am to 5 pm		
Reviews left voluntarily on a shop's website		

Task 4.2

A column of daily rainfall holds a reading of **487.0** mm.

(a) Give one reason to think it is an error.

(b) Give one reason it might be real.

(c) State what you would do before deciding, and why deleting it immediately would be wrong.

Extension 4.3

A council publishes every noise complaint it received, with the street name and the date, and no names. Explain who could still be identified, and which of the five quality factors is *not* the issue here.

Notes